

# PAC-MAN GAME

A fully playable Pac-Man built from scratch in Java — no database, just Swing and pure OOP.

● Week 7  
Complete

Java JDK 21

VS Code

Swing / JPanel

Frightened Mode

## WHAT WAS BUILT

### Frightened Mode

- ✓ Ghost.java tracks a frightened boolean and swaps speed on toggle
- ✓ setFrightened() slows ghosts from 1.5 to 0.75 tiles/frame
- ✓ draw() renders a blue circle with white outline when frightened
- ✓ resetToSpawn() returns an eaten ghost to its starting tile
- ✓ Ghost remembers its own startRow/startCol from construction

### Scoring & Triggers

- ✓ addPowerPellet() in ScoreManager resets the ghost-eat combo
- ✓ addGhostEaten() scores an escalating bonus: 200, 400, 800...
- ✓ GamePanel starts an 8-second frightened timer on pellet eaten
- ✓ checkGhostCollisions() detects Pac-Man/ghost tile overlap
- ✓ Frightened ghost eaten → bonus + reset; timer expiry reverts all

## KEY TECHNIQUES

### One Boolean Flag Drives Speed, Color, and Vulnerability

Ghost.java adds a single frightened field. setFrightened(true) does two things at once: switches speed from normalSpeed (1.5) to frightenedSpeed (0.75), and changes what draw() renders — a plain blue-and-white circle instead of the ghost's sprite. No other method needed to change: chooseDirection() and canMove() work exactly the same regardless of state, since they only read dx/dy/speed, not the frightened flag directly.

### Frightened Timer Lives in GamePanel, Not Ghost

Rather than each ghost tracking its own countdown, GamePanel keeps one shared frightenedTimer (480 frames ≈ 8 seconds at 60 FPS). Eating a power pellet resets it to full and flips all four ghosts frightened at once. Each frame, the timer ticks down; when it hits zero, every ghost reverts to normal together — matching how the arcade original synchronizes the frightened window across all ghosts rather than per-ghost.

### Escalating Ghost-Eat Bonus via a Combo Multiplier

ScoreManager tracks a ghostEatMultiplier that resets to zero every time a new power pellet is eaten. Each ghost eaten within that frightened window increments the multiplier and scores  $200 \times \text{multiplier}$  — so eating four ghosts in one pellet window scores 200, 400, 800, then 1600, rewarding aggressive back-to-back eating exactly like the original game's combo system.

### Collision Detection Reuses Existing Tile Coordinates

checkGhostCollisions() compares pacman.getRow()/getCol() against each ghost's getRow()/getCol() — coordinates every object already tracks for movement and wall-checking. No new collision system was needed. If a match is frightened, it's eaten (bonus + resetToSpawn()); a match while not frightened does nothing yet, since losing a life on contact is explicitly Week 8's job.

---

## SOURCE CODE

*Ghost.java (full)*

java

```

import java.awt.*;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.File;
import java.util.Random;

public class Ghost {
    protected int row, col;
    protected int startRow, startCol; // remembered for resetToSpawn()
    protected double x, y;
    protected int dx, dy;
    protected double normalSpeed = 1.5;
    protected double frightenedSpeed = 0.75;
    protected double speed = normalSpeed;
    protected BufferedImage sprite;
    protected Maze maze;
    protected PacMan pacman;
    protected Random random = new Random();
    protected boolean frightened = false;

    public Ghost(Maze maze, int startRow, int startCol, String spritePath, PacMan pacman) {
        this.maze = maze;
        this.row = startRow;
        this.col = startCol;
        this.startRow = startRow;
        this.startCol = startCol;
        this.x = col * Maze.TILE_SIZE;
        this.y = row * Maze.TILE_SIZE;
        this.dx = 0;
        this.dy = 0;
        this.pacman = pacman;
        try {
            sprite = ImageIO.read(new File(spritePath));
        } catch (Exception e) {
            System.out.println("Could not load sprite: " + spritePath);
        }
    }

    public void update() {
        if (isAligned()) chooseDirection();
        if (canMove(dx, dy)) {
            x += dx * speed;
            y += dy * speed;
        } else {
            x = col * Maze.TILE_SIZE;
            y = row * Maze.TILE_SIZE;
            dx = 0; dy = 0;
        }
        row = (int) Math.round(y / Maze.TILE_SIZE);
        col = (int) Math.round(x / Maze.TILE_SIZE);

        if (x < -Maze.TILE_SIZE) x = Maze.COLS * Maze.TILE_SIZE;
        if (x > Maze.COLS * Maze.TILE_SIZE) x = -Maze.TILE_SIZE;
    }

    protected int[] getTargetTile() {
        return new int[] { pacman.getRow(), pacman.getCol() };
    }

    protected void chooseDirection() {

```

```

int[][] directions = {{0,-1},{0,1},{-1,0},{1,0}};
int[] target = getTargetTile();

int bestDx = dx, bestDy = dy;
double bestDist = Double.MAX_VALUE;
boolean found = false;

for (int[] d : directions) {
    if (d[0] == -dx && d[1] == -dy) continue;
    if (!canMove(d[0], d[1])) continue;

    int nextRow = row + d[1];
    int nextCol = col + d[0];
    double dist = distance(nextRow, nextCol, target[0], target[1]);

    if (dist < bestDist) {
        bestDist = dist;
        bestDx = d[0];
        bestDy = d[1];
        found = true;
    }
}

if (!found) {
    for (int[] d : directions) {
        if (canMove(d[0], d[1])) {
            int nextRow = row + d[1];
            int nextCol = col + d[0];
            double dist = distance(nextRow, nextCol, target[0], target[1]);
            if (dist < bestDist) {
                bestDist = dist;
                bestDx = d[0];
                bestDy = d[1];
                found = true;
            }
        }
    }
}

if (found) {
    dx = bestDx;
    dy = bestDy;
}
}

protected double distance(int r1, int c1, int r2, int c2) {
    int dr = r1 - r2;
    int dc = c1 - c2;
    return Math.sqrt(dr * dr + dc * dc);
}

protected boolean canMove(int testDx, int testDy) {
    if (testDx == 0 && testDy == 0) return false;
    double nextX = x + testDx * speed;
    double nextY = y + testDy * speed;
    int checkRow = (int) Math.round(nextY / Maze.TILE_SIZE);
    int checkCol = (int) Math.round(nextX / Maze.TILE_SIZE);
    return !maze.isWall(checkRow, checkCol);
}

```

```

protected boolean isAligned() {
double diffX = x - col * Maze.TILE_SIZE;
double diffY = y - row * Maze.TILE_SIZE;
return Math.abs(diffX) < speed && Math.abs(diffY) < speed;
}

// ---- Frightened mode (Week 7) ----

public void setFrightened(boolean f) {
frightened = f;
speed = f ? frightenedSpeed : normalSpeed;
}

public boolean isFrightened() {
return frightened;
}

public void resetToSpawn() {
row = startRow;
col = startCol;
x = col * Maze.TILE_SIZE;
y = row * Maze.TILE_SIZE;
dx = 0;
dy = 0;
setFrightened(false);
}

public void draw(Graphics g) {
if (frightened) {
g.setColor(Color.BLUE);
g.fillOval((int) x, (int) y, Maze.TILE_SIZE, Maze.TILE_SIZE);
g.setColor(Color.WHITE);
g.drawOval((int) x, (int) y, Maze.TILE_SIZE, Maze.TILE_SIZE);
} else if (sprite != null) {
g.drawImage(sprite, (int) x, (int) y, Maze.TILE_SIZE, Maze.TILE_SIZE, null);
} else {
g.setColor(Color.RED);
g.fillOval((int) x, (int) y, Maze.TILE_SIZE, Maze.TILE_SIZE);
}
}

public int getRow() { return row; }
public int getCol() { return col; }
public double getX() { return x; }
public double getY() { return y; }
}

```

**ScoreManager.java (full)**

java

```

public class ScoreManager {
    private int score;
    private int lives;
    private int ghostEatMultiplier;

    public ScoreManager() {
        this.score = 0;
        this.lives = 3;
        this.ghostEatMultiplier = 0;
    }

    public void addDot() { score += 10; }

    public void addPowerPellet() {
        score += 50;
        ghostEatMultiplier = 0; // reset combo each time a new pellet is eaten
    }

    // Eating ghosts back-to-back scores more: 200, 400, 800, 1600...
    public void addGhostEaten() {
        ghostEatMultiplier++;
        score += 200 * ghostEatMultiplier;
    }

    public void loseLife() { lives--; }

    public int getScore() { return score; }
    public int getLives() { return lives; }
    public boolean isGameOver() { return lives <= 0; }
}

```

***GamePanel.java (full)***

java

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;

public class GamePanel extends JPanel implements Runnable {
    private Thread gameThread;
    private Maze maze;
    private PacMan pacman;
    private ScoreManager scoreManager;

    private RedGhost redGhost;
    private BrownGhost brownGhost;
    private BlueGhost blueGhost;
    private YellowGhost yellowGhost;
    private Ghost[] ghosts;

    private boolean gameOver = false;
    private boolean won = false;

    private static final int FRIGHTENED_DURATION = 480; // 8 seconds at 60 FPS
    private int frightenedTimer = 0;

    public GamePanel() {
        maze = new Maze();
        pacman = new PacMan(maze, 1, 1);
        scoreManager = new ScoreManager();

        redGhost = new RedGhost(maze, 10, 9, pacman);
        brownGhost = new BrownGhost(maze, 10, 10, pacman);
        blueGhost = new BlueGhost(maze, 10, 8, pacman, redGhost);
        yellowGhost = new YellowGhost(maze, 10, 7, pacman);
        ghosts = new Ghost[] { redGhost, brownGhost, blueGhost, yellowGhost };

        setBackground(Color.BLACK);
        setPreferredSize(new Dimension(
            Maze.COLS * Maze.TILE_SIZE,
            Maze.ROWS * Maze.TILE_SIZE + 40));
        setFocusable(true);

        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                switch (e.getKeyCode()) {
                    case KeyEvent.VK_UP: pacman.setDirection(0, -1); break;
                    case KeyEvent.VK_DOWN: pacman.setDirection(0, 1); break;
                    case KeyEvent.VK_LEFT: pacman.setDirection(-1, 0); break;
                    case KeyEvent.VK_RIGHT: pacman.setDirection(1, 0); break;
                }
            }
        });

        gameThread = new Thread(this);
        gameThread.start();
    }

    @Override
    public void run() {
        while (true) {
            if (!gameOver && !won) {

```

```

pacman.update();
for (Ghost ghost : ghosts) ghost.update();

int eaten = pacman.eatDot();
if (eaten == 2) scoreManager.addDot();
if (eaten == 3) {
    scoreManager.addPowerPellet();
    activateFrightenedMode();
}

updateFrightenedTimer();
checkGhostCollisions();

if (maze.countDots() == 0) won = true;
}
repaint();
try { Thread.sleep(16); }
catch (InterruptedException e) { e.printStackTrace(); }
}
}

private void activateFrightenedMode() {
    frightenedTimer = FRIGHTENED_DURATION;
    for (Ghost ghost : ghosts) ghost.setFrightened(true);
}

private void updateFrightenedTimer() {
    if (frightenedTimer > 0) {
        frightenedTimer--;
        if (frightenedTimer == 0) {
            for (Ghost ghost : ghosts) ghost.setFrightened(false);
        }
    }
}

private void checkGhostCollisions() {
    for (Ghost ghost : ghosts) {
        if (ghost.getRow() == pacman.getRow() && ghost.getCol() == pacman.getCol()) {
            if (ghost.isFrightened()) {
                scoreManager.addGhostEaten();
                ghost.resetToSpawn();
            }
            // Collision while NOT frightened (losing a life) is Week 8's job.
        }
    }
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

    g2.setFont(new Font("Arial", Font.BOLD, 16));
    g2.setColor(Color.WHITE);
    g2.drawString("SCORE", 10, 18);
    g2.setColor(Color.YELLOW);
    g2.drawString(String.valueOf(scoreManager.getScore()), 10, 34);

    g2.setColor(Color.WHITE);

```

```

g2.drawString("LIVES", 180, 18);
for (int i = 0; i < scoreManager.getLives(); i++) {
    g2.setColor(Color.YELLOW);
    g2.fillOval(180 + i * 22, 22, 14, 14);
}

g2.setColor(Color.WHITE);
g2.setFont(new Font("Arial", Font.BOLD, 16));
g2.drawString("PAC-MAN", 85, 25);

Graphics2D g2maze = (Graphics2D) g2.create();
g2maze.translate(0, 40);
maze.draw(g2maze);
pacman.draw(g2maze);
for (Ghost ghost : ghosts) ghost.draw(g2maze);
g2maze.dispose();

if (won) drawOverlay(g2, "YOU WIN!", Color.YELLOW);
else if (gameOver) drawOverlay(g2, "GAME OVER", Color.RED);
}

private void drawOverlay(Graphics2D g2, String message, Color color) {
    g2.setColor(new Color(0, 0, 0, 160));
    g2.fillRect(0, 0, getWidth(), getHeight());
    g2.setFont(new Font("Arial", Font.BOLD, 36));
    g2.setColor(color);
    FontMetrics fm = g2.getFontMetrics();
    int x = (getWidth() - fm.stringWidth(message)) / 2;
    int y = getHeight() / 2;
    g2.drawString(message, x, y);
}
}

```

App.java, GameWindow.java, Maze.java, PacMan.java, RedGhost.java, BrownGhost.java, BlueGhost.java, and YellowGhost.java are unchanged from Week 6.

## 10-WEEK PROGRESS

|                                          |    |
|------------------------------------------|----|
| Week 1 — Setup & Window                  | ✓  |
| Week 2 — Maze Design                     | ✓  |
| Week 3 — Pac-Man Character               | ✓  |
| Week 4 — Dot Collection & Scoring        | ✓  |
| Week 5 — Ghost Base Class                | ✓  |
| Week 6 — All Four Custom Ghosts          | ✓  |
| Week 7 — Power Pellets & Frightened Mode | ✓  |
| Weeks 8–10 — Polish + Ship               | 0% |

## UP NEXT

W  
8

### Collisions, Lives & Game States

Pac-Man touching a non-frightened ghost should cost a life and reset positions. A GameState enum (MENU, PLAYING, GAME\_OVER, WIN) will formalize the flow already sketched with won/gameOver flags.

*GameState.java · life-loss collision · respawn positions · win/lose flow*